

In [1]: `## 1.IMPORT`

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)
np.random.seed(42)
```

In [8]: `## 2. LOAD THE CSV FILE`

```
DATA_PATH = r"C:\Users\HP\Downloads\youtube.csv.zip" # <-- change to you

CATEGORY_ID_MAP = {
    1: 'Film & Animation', 2: 'Autos & Vehicles', 10: 'Music', 15: 'Pets',
    17: 'Sports', 18: 'Short Movies', 19: 'Travel & Events', 20: 'Gaming',
    21: 'Videoblogging', 22: 'People & Blogs', 23: 'Comedy', 24: 'Entertainment',
    25: 'News & Politics', 26: 'Howto & Style', 27: 'Education',
    28: 'Science & Technology', 29: 'Nonprofits & Activism', 30: 'Movies'
}

def parse_trending_date(series):
    """Handles Kaggle's legacy 'yy.dd.mm' format and standard ISO dates."
    s = series.astype(str)
    for fmt in ('%y.%d.%m', '%Y-%m-%d', '%Y-%m-%dT%H:%M:%S'):
        try:
            return pd.to_datetime(s, format=fmt, errors='raise')
        except (ValueError, TypeError):
            continue
    return pd.to_datetime(s, errors='coerce')

def load_youtube_csv(path):
    df = pd.read_csv(path, encoding='utf-8', on_bad_lines='skip')
    df['publish_date'] = pd.to_datetime(df['publish_date'], errors='coerce')
    df['trending_date'] = parse_trending_date(df['trending_date'])
    df['category'] = df['category_id'].map(CATEGORY_ID_MAP).fillna('Other')
    return df

df = load_youtube_csv(DATA_PATH)
print(f"Loaded {df.shape[0]:,} rows, {df.shape[1]} columns from {DATA_PATH}")
print(f"Unparsed trending_date: {df['trending_date'].isna().sum()}, unparsed publish_time: {df['publish_time'].isna().sum()}")
df.head(15)
```

Loaded 161,470 rows, 19 columns from C:\Users\HP\Downloads\youtube.csv.zip
Unparsed trending_date: 0, unparsed publish_time: 0

C:\Users\HP\AppData\Local\Temp\ipykernel_22304\1274225892.py:23: UserWarning: Parsing dates in %d/%m/%Y format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to silence this warning.

```
df['publish_date'] = pd.to_datetime(df['publish_date'], errors='coerce',
utc=True).dt.tz_localize(None)
```

Out[8]:

	index	video_id	trending_date	title	channel_title	category
0	0	2kyS6SvSYSE	2017-11-14	WE WANT TO TALK ABOUT OUR MARRIAGE	CaseyNeistat	
1	1	1ZAPwfrtAFY	2017-11-14	The Trump Presidency: Last Week Tonight with J...	LastWeekTonight	
2	2	5qpjK5DgCt4	2017-11-14	Racist Superman Rudy Mancuso, King Bach & Le...	Rudy Mancuso	
3	3	puqaWrEC7tY	2017-11-14	Nickelback Lyrics: Real or Fake?	Good Mythical Morning	
4	4	d380meD0W0M	2017-11-14	I Dare You: GOING BALD!?	nigahiga	
5	5	gHZ1Qz0KiKM	2017-11-14	2 Weeks with iPhone X	iJustine	
6	6	39idVpFF7NQ	2017-11-14	Roy Moore & Jeff Sessions Cold Open - SNL	Saturday Night Live	
7	7	nc99ccSXST0	2017-11-14	5 Ice Cream Gadgets put to the Test	CrazyRussianHacker	
8	8	jr9QtXwC9vc	2017-11-14	The Greatest Showman Official Trailer 2 [HD]...	20th Century Fox	
9	9	TUmyygCMMGA	2017-11-14	Why the rise of the robots won't mean the en...	Vox	

	index	video_id	trending_date	title	channel_title	category
10	10	9wRQljFNDW8	2017-11-14	Dion Lewis' 103-Yd Kick Return TD vs. Denver! ...	NFL	
11	11	VifQljit6A0	2017-11-14	(SPOILERS) 'Shiva Saves the Day' Talked About ...	amc	
12	12	5E4ZBSInqUU	2017-11-14	Marshmello - Blocks (Official Music Video)	marshmello	
13	13	GgVmn66oK_A	2017-11-14	Which Countries Are About To Collapse?	NowThis World	
14	14	TaTleo4cOs8	2017-11-14	SHOPPING FOR NEW FISH!!!	The king of DIY	

```
In [11]: ## 3.CLEAN & ENGINEER FEATURES

df = df.sort_values('trending_date').drop_duplicates(subset='video_id', k
df = df.dropna(subset=['publish_date', 'trending_date'])
df = df[df['views'] > 0]

for col in ['likes', 'dislikes', 'comment_count']:
    if col in df.columns:
        df[col] = df[col].clip(lower=0)

df['tags_count'] = df['tags'].fillna(['none']).apply(lambda t: 0 if t ==
df['title_length'] = df['title'].fillna('').str.len()
#df['description_length'] = df['description'].fillna('').astype(str).str
df['days_to_trend'] = (df['trending_date'] - df['publish_date']).dt.days.
df['publish_hour'] = df['publish_date'].dt.hour
df['publish_dow'] = df['publish_date'].dt.dayofweek
df['engagement_rate'] = (df['likes'] + df['comment_count']) / df['views']
df['like_dislike_ratio'] = df['likes'] / df['dislikes'].replace(0, 1)
df['log_views'] = np.log10(df['views'].clip(lower=1))

channel_counts = df['channel_title'].value_counts()
df['channel_video_count'] = df['channel_title'].map(channel_counts)
df['channel_tier'] = pd.qcut(df['channel_video_count'].rank(method='first

for col in ['comments_disabled', 'ratings_disabled']:
    if col in df.columns:
        df[col] = df[col].astype(bool).astype(int)
    else:
```

```
df[col] = 0

print("Missing values remaining:")
print(df.isna().sum()[df.isna().sum() > 0])
df[['views', 'likes', 'comment_count', 'engagement_rate', 'days_to_trend'
```

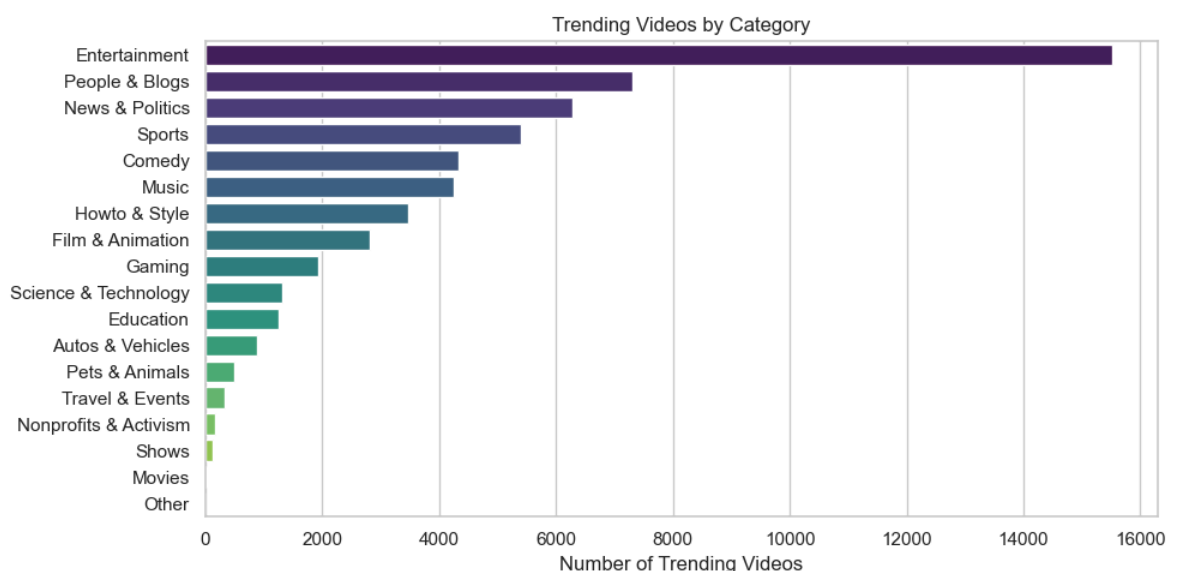
Missing values remaining:
Series([], dtype: int64)

Out[11]:

	views	likes	comment_count	engagement_rate	days_to_trend
count	5.588600e+04	5.588600e+04	55886.000000	55886.000000	55886.000000
mean	3.020267e+05	1.120424e+04	1521.192427	0.045948	1.64678
std	9.117620e+05	4.596806e+04	8249.640302	0.045432	4.4332
min	2.740000e+02	0.000000e+00	0.000000	0.000000	0.00000
25%	2.471225e+04	4.270000e+02	74.000000	0.011484	1.00000
50%	9.608300e+04	2.133000e+03	322.000000	0.030501	1.00000
75%	2.810418e+05	7.753750e+03	1095.000000	0.068129	1.00000
max	4.766929e+07	3.880074e+06	733371.000000	0.528285	60.00000

In [12]: *## 4. EDA*

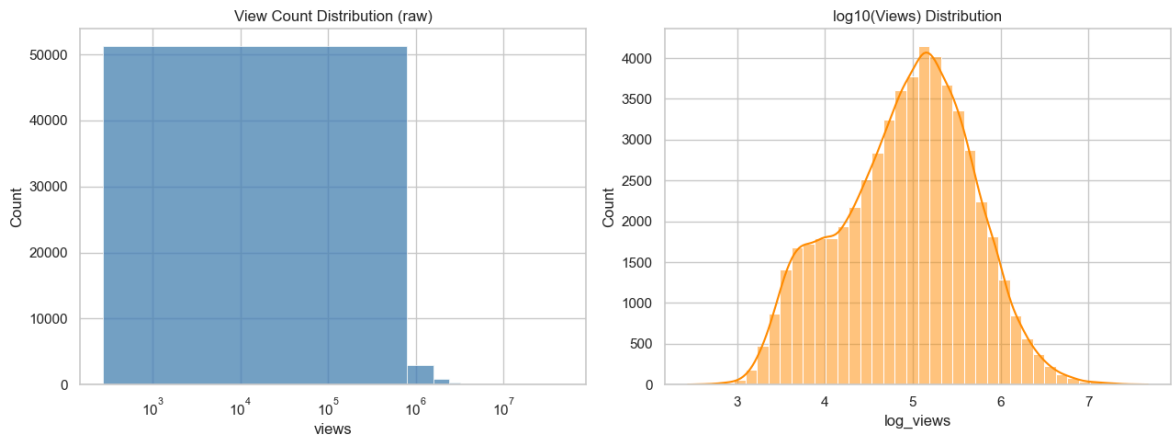
```
# 4.1 Category counts
cat_counts = df['category'].value_counts()
plt.figure(figsize=(10, 5))
sns.barplot(x=cat_counts.values, y=cat_counts.index, hue=cat_counts.index)
plt.title('Trending Videos by Category')
plt.xlabel('Number of Trending Videos'); plt.ylabel('')
plt.tight_layout(); plt.show()
```



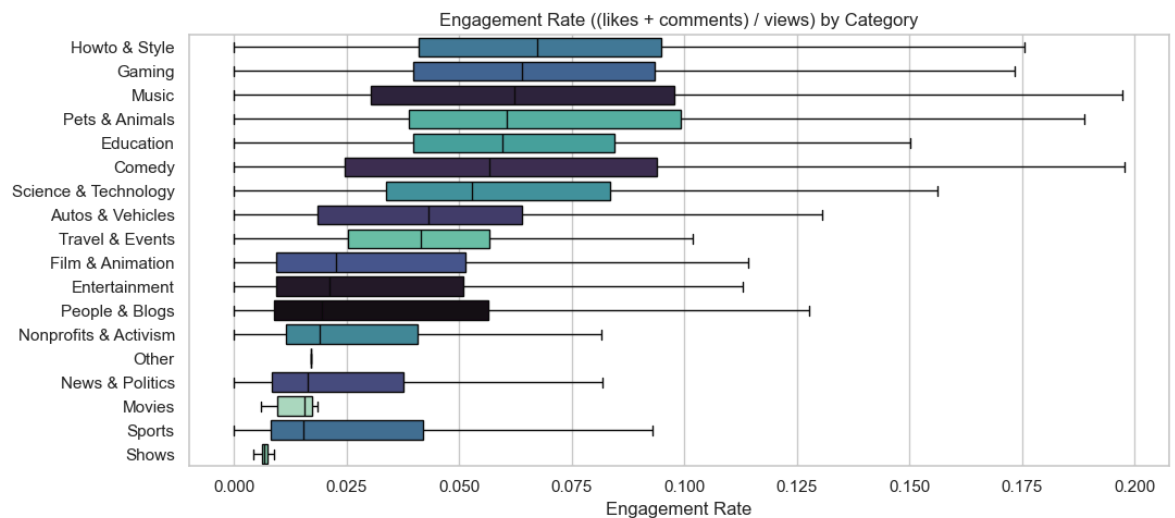
In [13]: *# 4.2 View distribution*

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.histplot(df['views'], bins=60, ax=axes[0], color='steelblue')
axes[0].set_title('View Count Distribution (raw)'); axes[0].set_xscale('log')
sns.histplot(df['log_views'], bins=40, ax=axes[1], color='darkorange', kd
```

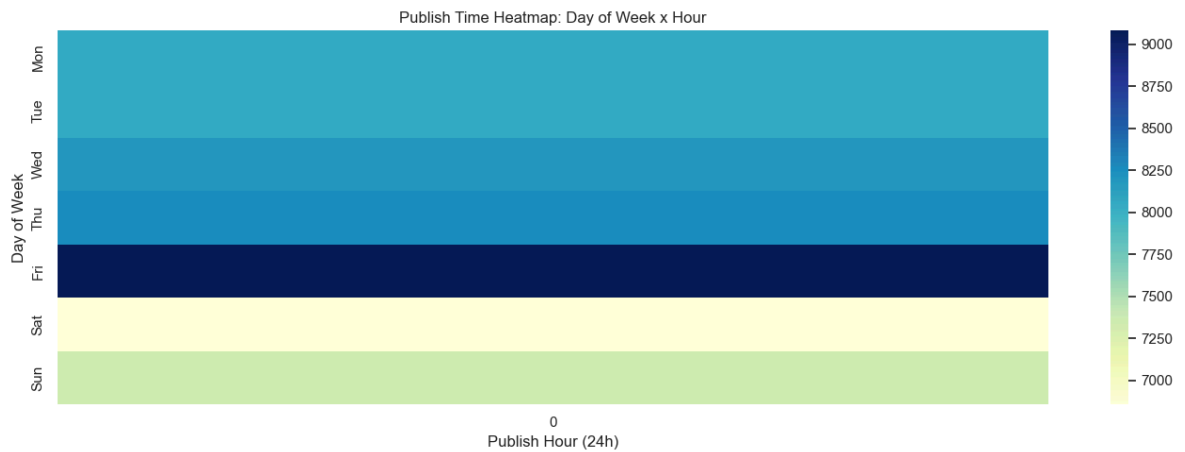
```
axes[1].set_title('log10(Views) Distribution')
plt.tight_layout(); plt.show()
```



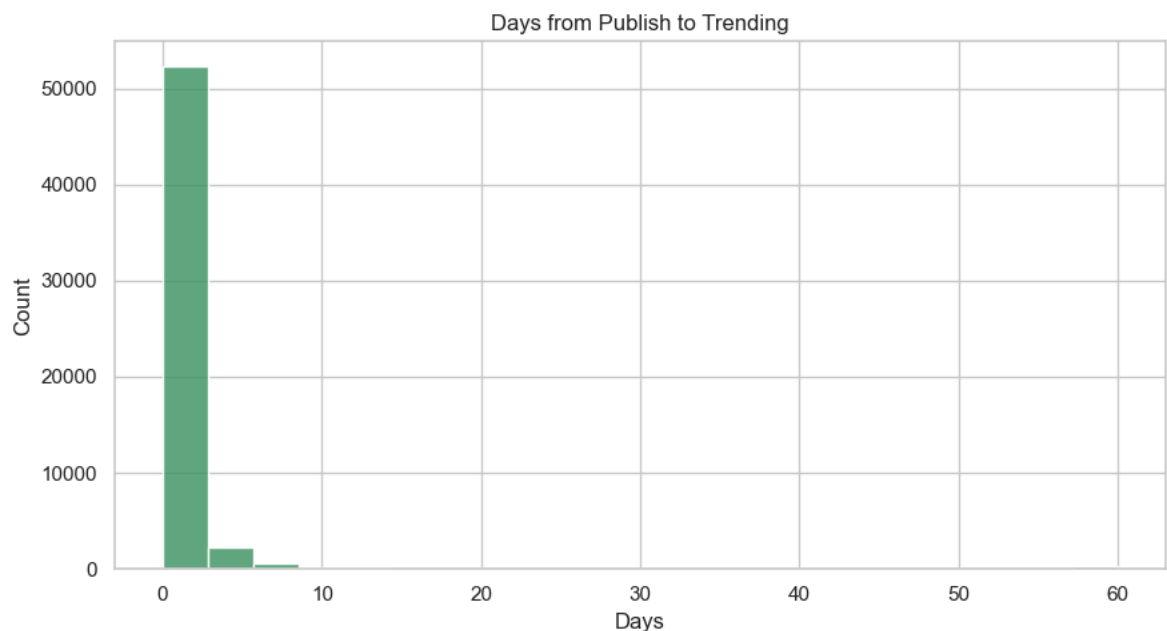
```
In [14]: # 4.3 Engagement rate by category
order = df.groupby('category')['engagement_rate'].median().sort_values(ascending=True)
plt.figure(figsize=(11, 5))
sns.boxplot(data=df, x='engagement_rate', y='category', order=order, hue=order,
            showfliers=False, palette='mako', legend=False)
plt.title('Engagement Rate ((likes + comments) / views) by Category')
plt.xlabel('Engagement Rate'); plt.ylabel('')
plt.tight_layout(); plt.show()
```



```
In [15]: # 4.4 Publish time heatmap
dow_labels = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
pivot = df.pivot_table(index='publish_dow', columns='publish_hour', value='engagement_rate')
plt.figure(figsize=(14, 5))
sns.heatmap(pivot, cmap='YlGnBu', yticklabels=dow_labels)
plt.title('Publish Time Heatmap: Day of Week x Hour')
plt.xlabel('Publish Hour (24h)'); plt.ylabel('Day of Week')
plt.tight_layout(); plt.show()
```

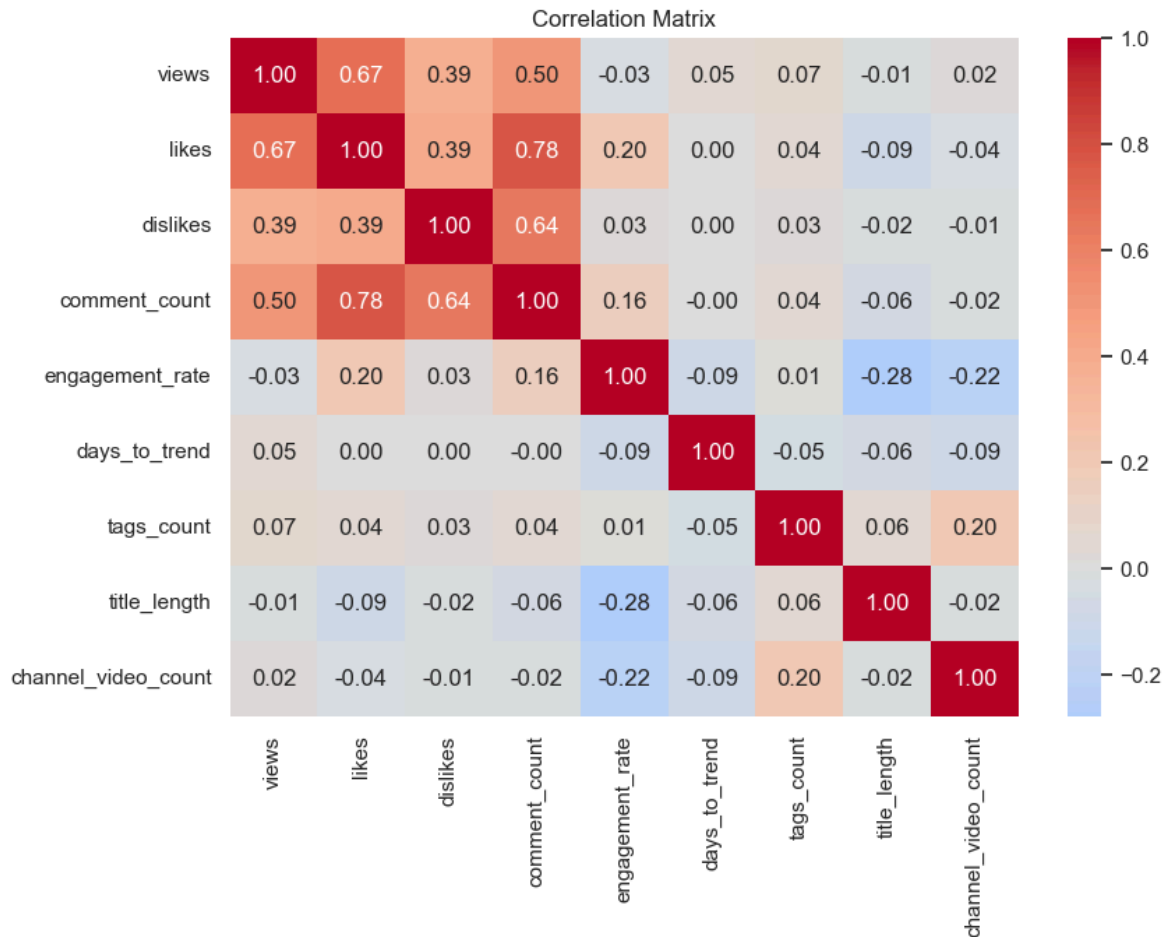


```
In [16]: # 4.5 Days to trend
plt.figure(figsize=(9, 5))
sns.histplot(df['days_to_trend'], bins=21, color='seagreen')
plt.title('Days from Publish to Trending'); plt.xlabel('Days')
plt.tight_layout(); plt.show()
print(df['days_to_trend'].describe())
```

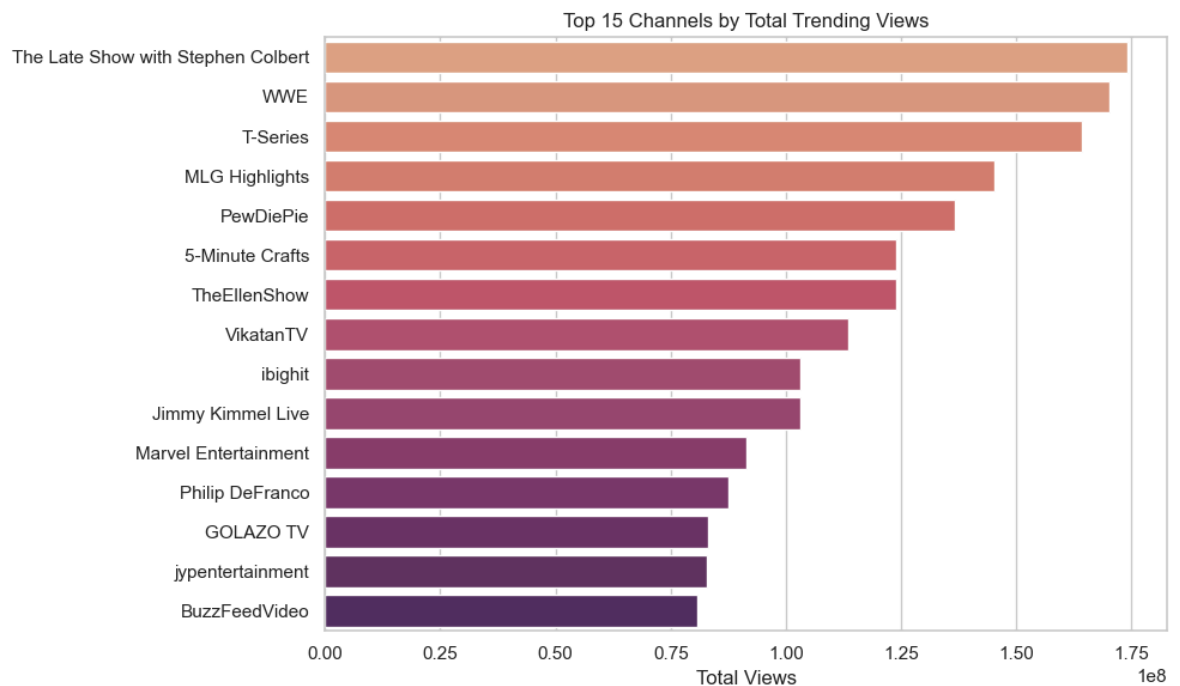


```
count    55886.000000
mean         1.646781
std         4.433210
min          0.000000
25%          1.000000
50%          1.000000
75%          1.000000
max         60.000000
Name: days_to_trend, dtype: float64
```

```
In [17]: # 4.6 Correlation matrix
num_cols = ['views', 'likes', 'dislikes', 'comment_count', 'engagement_ra
            'tags_count', 'title_length', 'description_length', 'channel_
num_cols = [c for c in num_cols if c in df.columns]
plt.figure(figsize=(9, 7))
sns.heatmap(df[num_cols].corr(), annot=True, fmt='.2f', cmap='coolwarm',
plt.title('Correlation Matrix')
plt.tight_layout(); plt.show()
```



```
In [18]: # 4.7 Top 15 channels
top_channels = df.groupby('channel_title')['views'].sum().sort_values(asc
plt.figure(figsize=(10, 6))
sns.barplot(x=top_channels.values, y=top_channels.index, hue=top_channels
plt.title('Top 15 Channels by Total Trending Views')
plt.xlabel('Total Views'); plt.ylabel('')
plt.tight_layout(); plt.show()
```




```

In [19]: ## 5.FORECASTING MODEL A-- PREDICTING VIEWS

feature_cols = ['title_length', 'tags_count', 'channel_tier', 'publish_ho
               'category', 'comments_disabled', 'ratings_disabled']
target_col = 'log_views'

X = df[feature_cols]
y = df[target_col]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

preprocess = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), ['category'])
], remainder='passthrough')

models = {
    'Linear Regression': Pipeline([('pre', preprocess), ('model', LinearR
    'Random Forest': Pipeline([('pre', preprocess),
                              ('model', RandomForestRegressor(n_estimat
                              random_s

    'Gradient Boosting': Pipeline([('pre', preprocess),
                                   ('model', GradientBoostingRegressor(n

})

results = []
fitted = {}
for name, pipe in models.items():
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)
    rmse = mean_squared_error(y_test, pred) ** 0.5
    mae = mean_absolute_error(y_test, pred)
    r2 = r2_score(y_test, pred)
    results.append({'model': name, 'RMSE (log10 views)': rmse, 'MAE': mae
    fitted[name] = pipe

results_df = pd.DataFrame(results).sort_values('R2', ascending=False)
results_df

```

```

Out[19]:

```

	model	RMSE (log10 views)	MAE	R2
1	Random Forest	0.644464	0.505812	0.249648
2	Gradient Boosting	0.670858	0.534896	0.186928
0	Linear Regression	0.698565	0.563994	0.118379

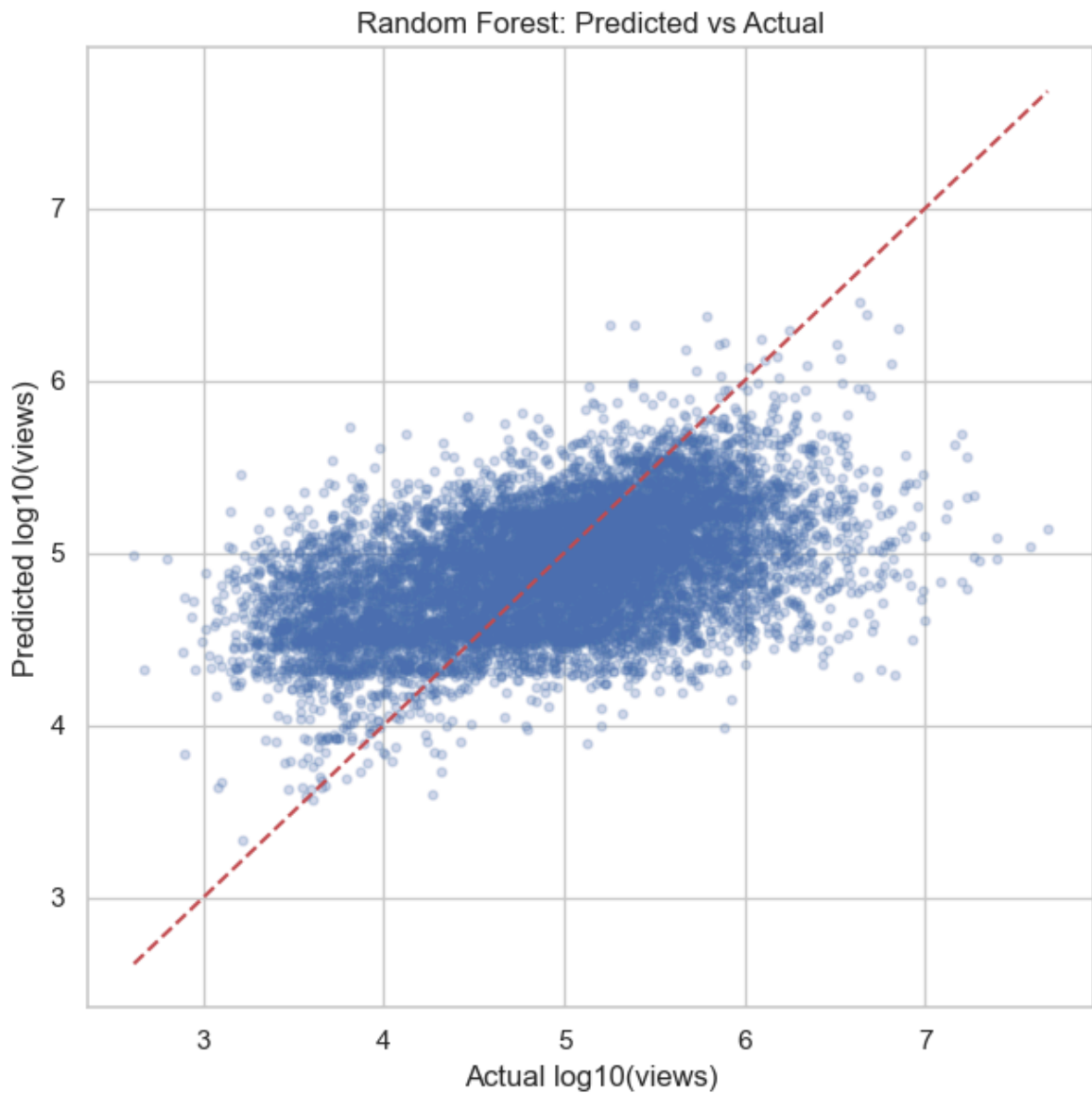
```

In [20]: # Predicted vs actual for best model
best_name = results_df.iloc[0]['model']
best_model = fitted[best_name]
pred_best = best_model.predict(X_test)

plt.figure(figsize=(6.5, 6.5))
plt.scatter(y_test, pred_best, alpha=0.25, s=12)
lims = [min(y_test.min(), pred_best.min()), max(y_test.max(), pred_best.m
plt.plot(lims, lims, 'r--', linewidth=1.5)
plt.xlabel('Actual log10(views)'); plt.ylabel('Predicted log10(views)')

```

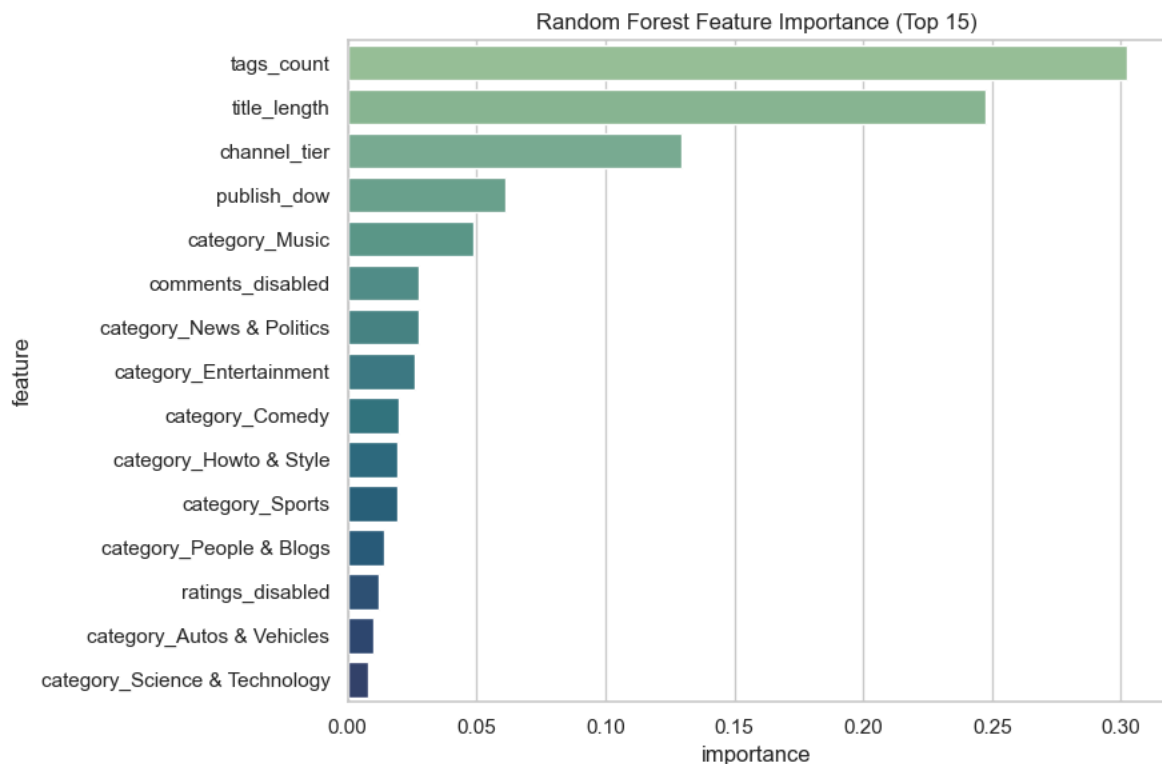
```
plt.title(f'{best_name}: Predicted vs Actual')
plt.tight_layout(); plt.show()
```



```
In [21]: # Feature importance
rf_pipe = fitted.get('Random Forest')
if rf_pipe is not None:
    ohe = rf_pipe.named_steps['pre'].named_transformers_['cat']
    cat_names = list(ohe.get_feature_names_out(['category']))
    passthrough_names = [c for c in feature_cols if c != 'category']
    all_names = cat_names + passthrough_names

    importances = rf_pipe.named_steps['model'].feature_importances_
    imp_df = pd.DataFrame({'feature': all_names, 'importance': importance
                          .sort_values('importance', ascending=False).head(15)})

    plt.figure(figsize=(9, 6))
    sns.barplot(data=imp_df, x='importance', y='feature', hue='feature',
    plt.title('Random Forest Feature Importance (Top 15)')
    plt.tight_layout(); plt.show()
```



```
In [22]: ## 6. FORECASTING MODEL B -- DAILY TRENDING VOLUME

daily = (df.groupby(df['trending_date'].dt.date)
          .size()
          .reset_index(name='trending_count'))
daily['trending_date'] = pd.to_datetime(daily['trending_date'])
daily = daily.sort_values('trending_date').reset_index(drop=True)

for lag in [1, 2, 3, 7]:
    daily[f'lag_{lag}'] = daily['trending_count'].shift(lag)
daily['dow'] = daily['trending_date'].dt.dayofweek
daily['rolling_mean_7'] = daily['trending_count'].shift(1).rolling(7).mean()

ts_df = daily.dropna().reset_index(drop=True)
split_idx = int(len(ts_df) * 0.8)
train_ts, test_ts = ts_df.iloc[:split_idx], ts_df.iloc[split_idx:]

ts_features = ['lag_1', 'lag_2', 'lag_3', 'lag_7', 'dow', 'rolling_mean_7']
ts_model = GradientBoostingRegressor(n_estimators=200, max_depth=2, learning_rate=0.1)
ts_model.fit(train_ts[ts_features], train_ts['trending_count'])
ts_pred = ts_model.predict(test_ts[ts_features])

ts_rmse = mean_squared_error(test_ts['trending_count'], ts_pred) ** 0.5
ts_mae = mean_absolute_error(test_ts['trending_count'], ts_pred)
print(f"Daily trending-count forecast - RMSE: {ts_rmse:.2f}, MAE: {ts_mae:.2f}")

plt.figure(figsize=(13, 5))
plt.plot(train_ts['trending_date'], train_ts['trending_count'], label='Train')
plt.plot(test_ts['trending_date'], test_ts['trending_count'], label='Test')
plt.plot(test_ts['trending_date'], ts_pred, label='Forecast', color='crimson')
plt.title('Daily Trending Video Count: Actual vs Forecast')
plt.xlabel('Date'); plt.ylabel('Trending videos / day')
plt.legend(); plt.tight_layout(); plt.show()
```

Daily trending-count forecast - RMSE: 59.40, MAE: 44.21

